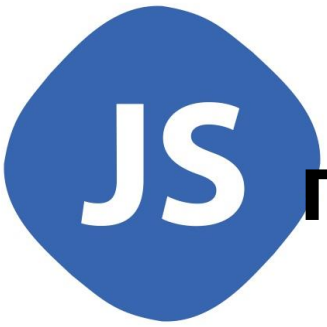


Переменные и константы

- Объявление переменной
- Имена переменных
- Константы
- Вывод в документ
- Консоль браузера
- Повторное объявление переменных
- Строгий режим
- Кеширование файлов

Объявление переменной

JavaScript-сценарию обычно нужно работать с информацией. Информацией веб-приложения может быть все, что угодно. Например:

- права доступа текущего пользователя;
- внешний вид отображаемой страницы;
- выбираемые пользователем товары и корзина покупок;
- критерии фильтрации и сортировки выводимых данных;
- и многое другое…

Переменные используются для хранения этой информации и являются основным понятием любого языка программирования.

Переменная — это именованная область памяти, в которой мы можем хранить какие-либо данные, например, строки или числа. Переменные похожи на условный контейнер в памяти компьютера. Этим контейнерам можно присваивать имена и впоследствии использовать эти имена для извлечения и изменения хранимых данных.

Прежде чем использовать переменную в JavaScript, ее необходимо **объявить**.





Объявление с помощью ключевого слова var

Для объявления или, другими словами, создания переменной используется ключевое слово **var** (от variable — переменная):

```
var myLogin;
var age;
```

После объявления переменной ей можно присвоить некоторое значение, это называется **инициализацией** переменной:

```
var myLogin;
myLogin = "master"; // сохраним в переменной строку
// можно совместить объявление переменной и ее инициализацию
var age = 33; // сохраним в переменной число
```

Значение переменной сохраняется в соответствующей именованной области памяти и в дальнейшем доступ к этой области осуществляется при обращении по имени переменной. Если переменную не инициализировать, то ее начальное значение остается неопределенным **undefined**.

example_1. Объявление переменных с помощью var

```
<script>
	// объявляем переменную myLogin
	var myLogin;
	// инициализируем переменную текстовым значением
	myLogin = "master";
	// объявляем переменную age
	var age;
	// выводим значения переменных
	alert(myLogin); // выведет содержимое переменной
	alert(age); // выведет undefined
</script>
```





При желании можно объявить сразу несколько переменных, разделив их запятыми.

```
// объявление нескольких переменных
var email, pwd, role;
```

Можно совместить объявление и инициализацию нескольких переменных.

```
// объявление и инициализация нескольких переменных
var email = 'master@mail.ru', pwd = '12345', role = 'admin';
```

Объявление (declared) переменной означает её поименование (тип в JavaScript определяется динамически). Если переменная объявлена без начального значения, то она еще не готова полностью к работе.

Инициализация (initialization) переменной означает, что переменная объявлена, ей присвоено начальное значение и она запущена в работу.

Без присвоения начального значения переменная просто объявлена, а с начальным значением она еще и инициализирована.

Особенности использования var:

- **var** не препятствует объявлению переменных с одинаковым именем;
- область видимости переменных **var** ограничивается либо функцией, либо, если переменная глобальная, то скриптом. Такие переменные доступны за пределами блока;
- если блок кода находится внутри функции, то **var** становится локальной переменной в этой функции.

> ***Важно***. Более подробно переменные **var** будут рассмотрены при изучении **управляющих конструкций**. Сейчас достаточно понять сопровождающие урок примеры кода.

Объявление с помощью ключевого слова let

Ключевое слово **let** появилось в ECMAScript 6, его, упрощённо, можно назвать блочной версией var.



```
let msg = 'Hello';
```

Ключевое слово **let** – почти то же самое, что и **var**. Область видимости переменных, объявленных с помощью ключевого слова **let**, ограничивается блоком, оператором или выражением, в котором оно объявлено, а также вложенными блоками.

> ***Примечание***. Тестирование различий между переменными **let** и **var**, будет проведено нами позже, пока эти различия не имеют для нас решающего значения.

example_2. Объявление переменных с помощью let

```
<script>
	// в простых примерах ничего не меняется
	// объявляем и инициализируем переменную myLogin
	let myLogin = "master";
	// объявляем переменную age
	let age;
	// выводим значения переменных
	alert(myLogin); // выведет содержимое переменной
	alert(age); // выведет undefined
</script>
```

Особенности использования let:

- **let** не позволяет обратиться к переменной до её объявления;
- **let** препятствует объявлению в одной области видимости переменных (и/или функций) с одинаковым именем;
- **let** является блочной или локальной (видна только внутри блока с областью видимости, ограниченной текущим блоком кода).

> ***Важно***. Более подробно переменные **let** будут рассмотрены при изучении



управляющих конструкций. Сейчас достаточно понять сопровождающие урок примеры кода.

Имена переменных

Имена переменных могут начинаться с одного из следующих символов:

- буква в верхнем или нижнем регистре: **a-z**, **A-Z**
- символ нижнего подчеркивания: **_**
- знак доллара: **$**

Вслед за первым символом имени переменной можно использовать последовательность букв, цифр и символа подчеркивания без пробелов и знаков препинания.

Поскольку JavaScript чувствителен к регистру, буквы включают символы от **A** до **Z** (верхний регистр) и символы от **a** до **z** (нижний регистр).

Примеры правильных имен переменных:

- `var myName;`
- `var my_adress;`
- `var _x;`
- `var tel512_7456;`

Примеры неправильных имен переменных:

- `var 512group; // начало не может быть цифрой`
- `var my-adress; // дефис '-' не является разрешенным символом`
- `var my adress; // пробел не является разрешенным символом`

При выборе имен переменных нельзя использовать **ключевые слова**, то есть слова, используемые в определениях конструкций языка. Например, нельзя выбирать слова **var**, **return**, **class**, **const**, **true**, **false**, **function** и ряд других.

Например, следующий сценарий выдаст синтаксическую ошибку:



```
var class = 15; // ошибка
```

> ***К сведению***. Имя переменной должно быть понятным и максимально чётко соответствовать хранимым в ней данным.

Существуют проверенные опытом **правила** именования переменных:

- имена, состоящие из одной буквы, равно как и имена, не несущие смысловой нагрузки, можно применять только в том случае, если они используются в небольшом фрагменте кода и их применение очевидно;
- используйте составные имена из нескольких слов, чтобы как можно точнее описать назначение переменных.
- возможны два способа **объединения слов** для создания имени: использование "верблюжьей" нотации (**camelCase**) или запись слов через символ нижнего подчеркивания (**snake_case**).
- при выборе имен применяйте только **английский** язык, т.к. среди разработчиков всего мира принято использовать английский язык для имён переменных (*впрочем, если вы планируете покорить мир своими программами…, ведь Стив Джобс тоже не стал бы тем, кем он стал, если бы действовал в шаблонах чужих правил;*).

Константы

Кроме переменных в JavaScript можно создавать константы. **Константа** – это переменная, которая не изменяется в процессе выполнения скрипта. Создать именованную константу можно используя ключевое слово **const**.

Для примера создадим константу, в которой будет хранится значение числа пи:

```
const pi = 3.14;
```

Как правило константы записываются большими буквами, чтобы их можно было отличить от обычных переменных:

```
const PI = 3.14;
```



```
const AGE = 18;
```

> ***К сведению***. Имена констант пишутся буквами в **верхнем регистре**, через нижнее подчёркивание (если необходимо). В остальном синтаксис идентификатора константы является таким же, как идентификатора обычной переменной.

Имена констант не должны совпадать с именами переменных, объявленных в той же области видимости, например, следующий сценарий выдаст синтаксическую ошибку.

```
const MY_LOGIN = "master";
var MY_LOGIN = "master"; // ошибка
```

Попытки изменить значение константы также приведут к ошибке.

```
const FLAG = 1;
FLAG = 2; // ошибка
alert(FLAG); // не сработает
```

Особенности использования констант:

- константа должна быть сразу инициализирована (а не только объявлена);
- константа не может быть переопределена.

Вывод в документ

С помощью средств JavaScript можно управлять элементами HTML страницы, например, выводить в них текст. Подробнее возможности манипулирования элементами страницы будем рассматривать в теме, посвященный работе с DOM.



Иногда, однако, в **отладочных** целях требуется вывести какой-нибудь текст в окно браузера. Это делается с помощью команды **document.write**. Вывод в браузер может выглядеть следующим образом:

```
// вывод простого текста
document.write ("Изучаем JavaScript!");
// вывод числового значения
document.write (12345);
// вывод текста из переменной
var email = 'master@mail.ru';
document.write (email);
```

Приведу еще несколько примеров вывода в браузер с помощью document.write.

example_3. Примеры вывода в браузер

```
<script>
	// вывод текста, заключенного в HTML-теги
	document.write('<b>text1</b>');
	// или так
	let str = 'text2';
	document.write('<b>' + str + '</b>');
	// вывод текста с переносом строки
	document.write('text3 <br> text4');
	// вывод большого объема текста с переносом строки
	document.write('text5<br>');
	document.write('text6<br>');
	document.write('text7<br>');
	// или так
	document.write('text8');
	document.write('<br>');
```

```
    document.write('text9');
    // или так
    document.write('text10', '<br>', 'text11');
</script>
```

Консоль браузера

Код уязвим для ошибок. Программист не робот, ему свойственно ошибаться, свойственно делать ошибки в коде. Но по умолчанию в браузере ошибки не видны. Если в JS-код закралась ошибка, она не будет видна обычному посетителю. Обычной возможностью вывода document.write мы не увидим, что именно пошло не так, и не сможем это исправить.

Для удобства программистов в каждом браузере есть специальная панель разработчика, в которую встроены так называемые **Инструменты разработки** (Developer tools или сокращённо — **devtools**).

С помощью специальных консольных команд вы можете выводить данные своих скриптов в консоль, это применяется для того, чтобы заниматься **отладкой программ**.

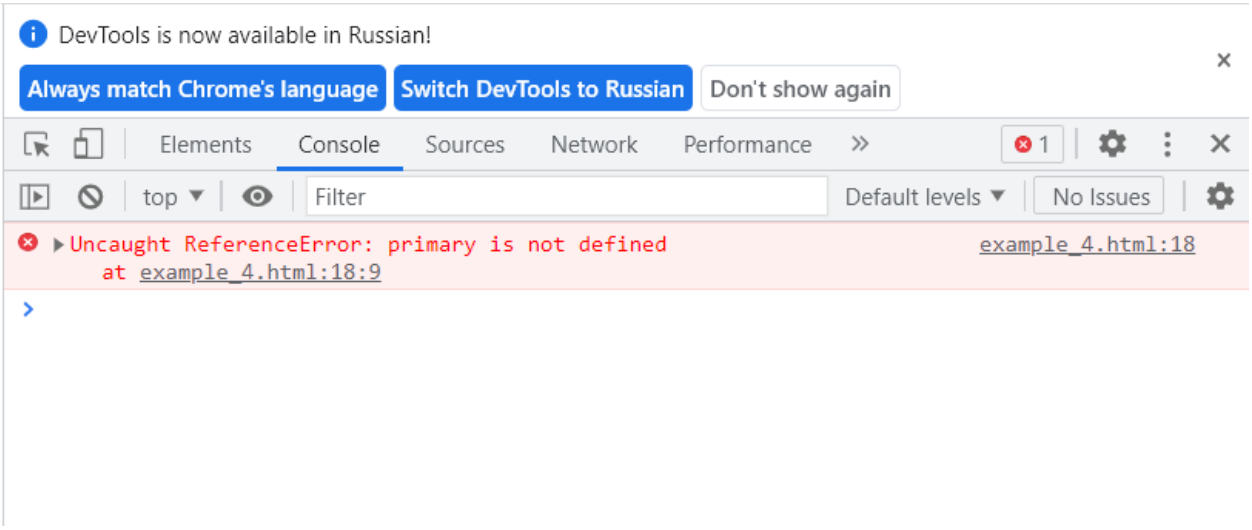
Для примера выведем что-нибудь в консоль. Запустите код сценария example_4.

example_4. Запуск консоли браузера

```
<script>
    // инициализируем переменную
    var message = "Добро пожаловать в мир JS!";
    // переменная primary ранее не была определена
    alert(primary);
</script>
```

Для вызова инструментов разработчика нажмите **F12**, в открывшейся панели вы чаще всего будете пользоваться двумя вкладками: **Elements** и **Console**. В первой вкладке вы можете получать информацию по структуре страницы, а во второй - отладочную информацию JavaScript. По умолчанию в инструментах разработчика откроется вкладка **Console** (Консоль).

Вкладка браузера Google Chrome выглядит приблизительно следующим образом.



Точный внешний вид инструментов разработки зависит от используемой версии Chrome. Время от времени некоторые детали изменяются, но в целом внешний вид остаётся примерно похожим на предыдущие версии.

В приведенном скриншоте консоли мы можем увидеть сообщение об ошибке, отрисованное красным цветом. В данном случае сообщение говорит о том, что скрипт содержит неизвестную переменную "primary".

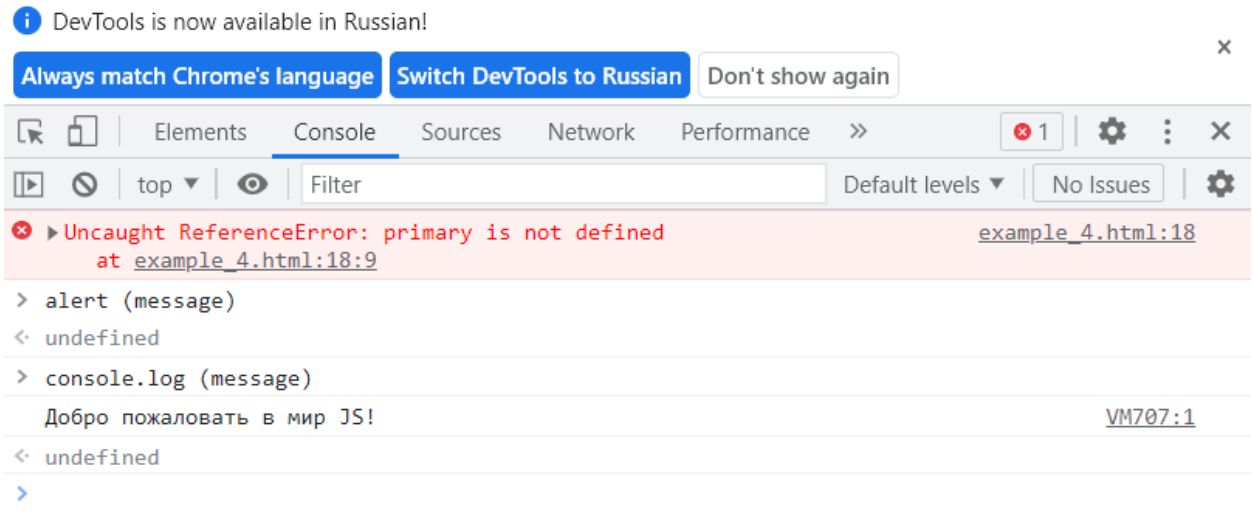
Справа присутствует ссылка на исходный код **example_4.html:18** с номером строки кода, в которой эта ошибка и произошла.

Под сообщением об ошибке находится синий символ **>**. Он обозначает командную строку, в ней вы можете редактировать и запускать JavaScript-команды.

Для запуска команды необходимо нажать **Enter**. Обычно при нажатии Enter введённая строка кода сразу выполняется. Чтобы перенести

строку, нажмите **Shift+Enter**. Таким образом, можно вводить многострочный JS-код.

В примере ниже я дважды вывел значение определенной в сценарии переменной **message**, сначала в диалоговом окне **alert**, затем используя рассматриваемый далее консольный метод **log**.



Объект **Console** служит для доступа к средствам отладки браузера. Работа с ним может отличаться в разных браузерах, но эти методы и свойства по факту поддерживаются всеми браузерами.

Приведу несколько примеров использования **методов**, доступных в объекте Console.

console.log()

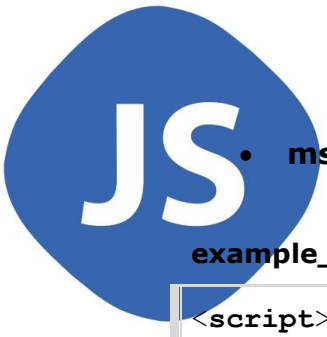
Выводит сообщение (логирует) в веб-консоль.

Синтаксис:

```
console.log(obj1 [, obj2, ..., objN]);
console.log(msg [, obj1, ..., objN]);
```

Параметры:

- **obj1**, ..., **objN** - список объектов для вывода.



- **msg** - строка, предваряющая вывод.

example_5. Тестируем console.log

```
<script>
	// определим переменную
	var msg = "Stop the World – I Want to Get Off";
	// определим массив
	var arr = new Array("php", "sql", "javascript", "css");
	// выведем данные в консоль
	console.log(msg);
	console.log(arr);
</script>
```

console.dir()

Способ просмотреть в консоли список свойств указанного JavaScript объекта. Вывод представлен в виде иерархического списка с возможностью просмотра содержимого дочерних объектов.

Синтаксис:

```
console.dir(obj);
```

Параметры:

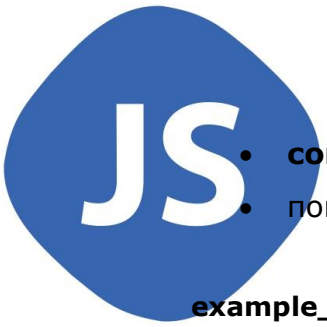
- **obj** - JavaScript-объект свойства которого нужно вывести.

Примечание:

Несколько заметок о разнице между console.dir() и console.log():

- **console.dir** может принимать только один аргумент, тогда как в console.log() мы можем передавать несколько аргументов. Если в console.dir() передается несколько аргументов, в консоль будет выведен только первый аргумент, а остальные аргументы будут проигнорированы;
- **console.log** выводит элемент как HTML-дерево;





- **console.dir** выводит элемент как JSON-объект;
- поведение методов может зависеть от браузера.

example_6. Тестируем console.dir

```
<script>
	// получим ссылку на список
	let web = document.querySelector("ul");
	// выведем список в консоль
	// смотрим разницу
	console.log("Список: ", web);
	console.dir(web);
</script>
```

console.info()

Выводит сообщения, содержащие некоторую информацию, в веб-консоль.

Синтаксис:

```
console.info(obj1 [, obj2, ..., objN]);
```

Параметры:

- **obj1**, ..., **objN** - список объектов для вывода. Строковое представление каждого из них добавляется в указанном порядке вывода.

example_7. Тестируем console.info

```
<script>
	// получим ссылку на список
	let web = document.querySelector("ul");
	// выведем список в консоль
	console.log("Список: ", web);
	console.info("Список: ", web);
```



```
    console.info("Список: ");
    console.dir(web);
</script>
```

Между командами **console.log** и **console.info** могут проявляться отличия при более глубоком погружении в программирование, в том числе в **node.js**. Нам эти возможности не понадобятся.

console.clear()

Очищает консоль. Содержимое консоли заменяется на информационное сообщение, которое может быть следующим: "Консоль очищена".

Синтаксис:

```
console.clear();
```

example_8. Тестируем console.clear

```
<script>
    // выводим что-либо в консоль
    console.log("Изучаем JavaScript вместе!");
    // очищаем консоль
    console.clear();
</script>
```

Эта возможность не является стандартной по этой причине ее поведение предсказать невозможно.

> ***К сведению***. На самом деле методов и возможностей объекта Console достаточно много. Для полного ознакомления смотрите справочные материалы.

Но на первых порах представленных возможностей будет вполне достаточно.



Повторное объявление переменных

Хотя различия между ключевыми словами **var** и **let** достаточно тонкие, и проявляются в написании сценариев, содержащих блоки кода, одно из них можно обнаружить уже сейчас.

Переменную, объявленную с помощью ключевого слова **var**, можно объявить повторно, такое переобъявление вполне допустимо.

example_9. Повторное объявление переменных var

```
<script>
	// тестируем поведение переменных с ключевым словом var
	// переменная с названием неплохого фильма
	var myFilm = "Зеленая миля";
	// переменная с названием советского фильма
	var myFilm = "Тот самый Мюнхгаузен";
	console.log (myFilm); // Тот самый Мюнхгаузен
</script>
```

Переменная, объявленная с помощью ключевого слова **let** может быть объявлена только один раз, повторное объявление вызывает ошибку.

example_10. Повторное объявление переменных let

```
<script>
	// тестируем поведение переменных с ключевым словом let
	// переменная с названием советского фильма
	let moreFilm = "Человек с бульвара Капуцинов";
	// переменная с названием неплохого фильма
	let moreFilm = "Криминальное чтиво";
	console.log (moreFilm);
	// Uncaught SyntaxError: Identifier 'moreFilm' has already been
```



```
declared
</script>
```

Переменную let следует объявлять только один раз и затем использовать её уже без ключевого слова let.

> ***Вывод.*** Отсюда **следует** простой вывод - советские фильмы лучшие и переопределить их чем-либо не получится! Или **не следует**… О чем это я?...

Строгий режим

Строгий режим (**strict mode**) — это важная часть современного JavaScript. Именно этот режим позволяет разработчикам пользоваться более ограниченным, чем стандартный, синтаксисом.

Семантика строгого режима отличается от традиционного нестрогого режима, который иногда называют "грязным" (**sloppy mode**).

> ***К сведению***. В "грязном" режиме синтаксические правила языка не так строги, а когда происходят некоторые ошибки, система никак не оповещает о них пользователя. То есть — ошибки могут быть **проигнорированы**, а код, в котором они допущены, сможет выполняться дальше.

Это способно привести к неожиданным результатам выполнения кода.

Строгий режим вносит в семантику JavaScript некоторые изменения. Он не даёт системе закрывать глаза на ошибки, выдавая соответствующие исключения. Это приводит к остановке выполнения программ.

Строгий режим, кроме того, помогает в написании программ, в которых нет недочётов, мешающих JS-движкам оптимизировать код. В строгом режиме запрещено использование элементов синтаксиса, которые могут получить особый смысл в будущих версиях языка.



Особенности применения строгого режима

Строгий режим можно применять к отдельным функциям или к целому скрипту. Его нельзя применить только к отдельным инструкциям или к блокам кода, заключённым в фигурные скобки.

> ***Примечание***. Наиболее ярко различия режимов проявляются при работе с блоками кодов, функциями, объектами. Сейчас важно понять, что такое строгий режим и как он влияет на код сценария.

Для того чтобы использовать строгий режим на уровне целого скрипта, в самое начало файла, до любых других команд, нужно поместить конструкцию "use strict" или 'use strict'.

```
<script>
      "use strict";
      // ...
```

Если в проекте имеются некоторые скрипты, в которых не используется строгий режим, и другие, в которых этот режим используется, тогда может случиться так, что эти скрипты окажутся объединены.

Это приведёт к тому, что код, который не предназначен для выполнения в строгом режиме, окажется в таком состоянии, когда система попытается выполнить его в строгом режиме. Возможно и обратное — код, написанный для строгого режима, попадёт в нестрогий режим. Поэтому лучше всего не смешивать "строгие" и "нестрогие" скрипты.

Преобразование "тихих" ошибок в исключения

Строгий режим влияет и на синтаксис кода, и на то, как код ведёт себя во время выполнения программы. Ошибки в коде преобразуются в исключения. То, что в нестрогом режиме тихо даёт сбой, в строгом вызывает сообщение об ошибке



"Тихие" ошибки преобразуются в строгом режиме в исключения. В нестрогом режиме на такие ошибки система явным образом не реагирует. В строгом же режиме наличие таких ошибок приводит к неработоспособности кода.

Так, благодаря этому сложно совершить ошибку случайного объявления глобальной переменной, так как переменные и константы в строгом режиме нельзя объявлять без использования директив **var**, **let** или **const**. В результате создание переменных без этих директив приведёт к неработоспособности программы.

Поясню на примере. Следующий фрагмент кода не выдаст никаких предупреждений и создаст глобальную переменную myVariable.

```
<script>
      myVariable = "It's my life";
</script>
```

Но такой код нельзя запустить в строгом режиме. Строгий режим защищает программиста от непреднамеренного создания глобальных переменных.

```
<script>
      'use strict';
      myVariable = "It's my life";
</script>
> Uncaught ReferenceError: myVariable is not defined
```

example_11. Применение режима use strict

```
<script>
      'use strict'; // включаем строгий режим
       // myVariable = "Test use strict";
      var myVariable = "Test use strict";
      console.log (myVariable);
```

```
</script>
```

Попытка выполнения любого кода, который, в обычном режиме, просто не работает, теперь приводит к выдаче исключения. В виде ошибок рассматриваются любые неправильные синтаксические конструкции, которые в нестрогом режиме просто игнорировались.

Кеширование файлов

Веб-кэш располагается между веб-сервером и клиентом и следит за входящими запросами, сохраняя при этом копии ответов — HTML-страниц, скриптов, изображений и прочих файлов (контент), для собственных нужд. Затем, если поступает другой запрос с аналогичным URL-адресом, кэш может использовать сохраненный прежде ответ, вместо повторного запроса к серверу.

Существует две основные причины, по которым используется веб-кэш:

1. **Уменьшение времени ожидания** — так как данные по запросу берутся из кэша (который располагается "ближе" к клиенту), требуется меньше времени для получения и отображения контента на стороне клиента. Это делает веб более отзывчивым.
2. **Снижение сетевого трафика** — повторное использование контента снижает объем данных, передаваемых клиенту.

> ***Важно***. Браузер может кэшировать подключенные JavaScript файлы. Это значит, что он сохраняет их у себя для повышения скорости загрузки сайта.

На практике это ведет к тому, что в какой-то момент при редактировании кода браузер будет применять предыдущую версию кода, а не вашу текущую. Для борьбы с таким поведением можно чистить кэш браузера.

Виды веб-кэшей:

- **Кэш браузера** (Browser cache).



Эта опция позволяет выделить область жесткого диска на вашем компьютере для хранения просмотренного ранее контента. Кэш браузера работает согласно довольно простым правилам. Он просто проверяет являются ли данные "свежими", обычно один раз за сессию (то есть, один раз в текущем сеансе браузера).

Этот кэш особенно полезен, когда пользователь нажимает кнопку "Назад" или кликает на ссылку, чтобы увидеть страницу, которую только что просматривал. Также, если вы используете одни и те же изображения навигации на вашем сайте, они будут выбираться из браузерного кэша почти мгновенно.

- **Прокси-кэш** (Proxy cache)

Прокси-кэш работает по аналогичному принципу, но в гораздо большем масштабе. Прокси обслуживают сотни или тысячи пользователей; большие корпорации и интернет-провайдеры часто настраивают их на своих файрволах или используют как отдельные устройства (intermediaries).

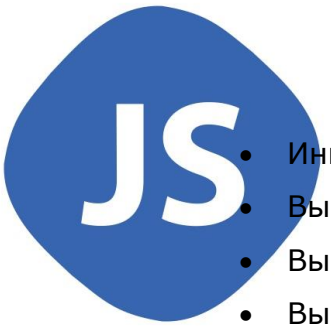
Прокси-кэши являются своего рода общей кэш-памятью (shared cache): вместо обслуживания одного человека, они работают с большим числом пользователей и поэтому очень хороши в сокращении времени ожидания и сетевого трафика. В основном, из-за того, что популярный контент запрашивается много раз.

- **Кэш-шлюз** (Gateway Cache)

Также известные как “реверсивные прокси-кэши” (reverse proxy cache) или "суррогаты" (surrogate cache) шлюзы тоже являются посредниками, но вместо того, чтобы использоваться системными администраторами для сохранения пропускной способности канала, они (шлюзы) обычно используются веб-мастерами для того, чтобы сделать их сайты более масштабируемыми, надежными и эффективными.

Сети доставки контента (content delivery networks, CDN) распространяют шлюзы по всему интернету (или некоторой его части) и отдают кэшированный контент заинтересованным веб-сайтам.

В **самостоятельных работах** закрепим пройденный материал урока:



- Инициализация и вывод переменных.
- Вывод контента в браузер.
- Вывод изображений в скрипте.
- Вывод данных в консоль.
- Отладка сценариев.

